

The 1D Hydraulic Calculation Process

of the LocalSWMM project (HydroCouple OpenSWMM engine)

Technical Reference Document

August 2026

Abstract

This document provides a complete, equation-level description of the one-dimensional (1D) hydraulic calculation process used by the LocalSWMM web application. All hydraulic computations are executed by the HydroCouple **OpenSWMM** engine, which is compiled to WebAssembly and runs inside the browser. The engine solves the de Saint-Venant equations on a node-link network. Four routing formulations are available: *steady flow*, *kinematic wave*, *dynamic wave* (the default), and an explicit *finite-volume* scheme. This document explains the conceptual model, the governing equations, the finite-difference schemes, the iterative solution procedure, the node-continuity and surcharge algorithms, the computation of non-conduit hydraulic structures, the cross-section geometry functions, boundary conditions, adaptive time stepping, and the mass-balance accounting. Every equation is transcribed from the engine source code (see the References for the file list) and cross-referenced to the engine's Reference Manuals where applicable.

Contents

1	Introduction	4
1.1	Scope	4
1.2	The node-link conceptual model	4
1.3	State variables and internal units	4
1.4	The four routing formulations	5
2	Hydraulic model components	5
2.1	Node data	5
2.1.1	Node surface area and volume functions	6
2.2	Link data	6
2.3	Conduit slope and length	7
2.4	Conduit lengthening (Courant stability)	7
3	Simulation orchestration	7
3.1	The overall time-stepping loop	7
3.2	Runoff and routing clocks	8
3.3	Lateral inflows	8
3.4	The routing step	8
4	Cross-section geometry	8
4.1	The four geometric relations	8
4.2	Manning and the section factor	9
5	Steady flow routing	9

6	Kinematic wave routing	9
6.1	Governing equations	9
6.2	Finite-difference scheme	9
6.3	Root finding	10
6.4	Storage nodes under kinematic wave	10
7	Dynamic wave routing: governing equations	10
7.1	The St. Venant equations	10
7.2	Finite-difference momentum equation	11
7.3	Node continuity	11
8	The dynamic wave solution algorithm	11
8.1	Overview: the Picard (successive-approximation) iteration	11
8.2	Convergence	12
8.3	Flow classification	12
8.4	Surface-area contributions	12
9	The link momentum kernel	13
9.1	Velocity, Froude number, and inertial damping	13
9.2	Upstream weighting	13
9.3	The six momentum terms	13
9.4	Flow limiting and post-processing	14
9.5	Force mains	14
10	Node continuity, flooding, and surcharge	14
10.1	Net flow and volume change	14
10.2	Free-surface update (EXPLICIT continuity)	15
10.3	Surcharge (EXTRAN) update	15
10.4	Semi-implicit node continuity	15
10.5	Flooding and ponding	15
11	Surcharge methods	16
11.1	EXTRAN (default)	16
11.2	Static Preissmann slot	16
11.3	Dynamic Preissmann slot	16
12	Non-conduit hydraulic structures	17
12.1	Pumps	17
12.2	Orifices	17
12.3	Weirs	17
12.4	Outlets	18
13	Outfall boundary conditions	18
14	Flow dividers	18
15	Adaptive time stepping and the Courant condition	18
15.1	Courant–Friedrichs–Lewy condition	18
15.2	Link-based step	19
15.3	Node-based step	19
15.4	Combining the constraints	19

16 Stability, convergence, and mass balance	19
16.1 Stability indicators	19
16.2 Convergence of the Picard loop	19
16.3 Routing mass balance	20
17 Options and defaults	20
18 References	20

1 Introduction

1.1 Scope

This document covers the *one-dimensional* hydraulic (flow-routing) calculations of the LocalSWMM project. LocalSWMM is a browser-based web application for stormwater and wastewater network modeling; it embeds the HydroCouple OpenSWMM engine (C++) compiled to WebAssembly. The 1D hydraulic component solves unsteady, gradually varied, one-dimensional flow through a network of *nodes* (junctions, storage units, dividers, outfalls) and *links* (conduits, pumps, orifices, weirs, outlets). The 2D overland-flow module and its 1D–2D coupling are out of scope except where they interact with the 1D solver (e.g. through lateral inflows and the ponded-area coupling).

The engine is a *distributed discrete-time simulation model*: it advances a state vector over a sequence of time steps,

$$\mathbf{X}_t = f(\mathbf{X}_{t-1}, \mathbf{I}_t, \mathbf{P}), \quad \mathbf{Y}_t = g(\mathbf{X}_t, \mathbf{P}), \quad (1)$$

where $\mathbf{X}$ is the state vector, $\mathbf{I}$ external inputs, $\mathbf{P}$ fixed parameters, and $\mathbf{Y}$ the outputs.

1.2 The node–link conceptual model

The conveyance portion of a drainage system is a network of nodes and links. **Nodes** are points that represent:

- **Junctions** — points where links join, with invert elevation, height-to-rim, optional surcharge (pressure) depth, and optional ponded surface area.
- **Outfalls** — terminal boundary nodes with a prescribed stage condition (free, normal, fixed, tidal, or time-series) and an optional flap gate.
- **Storage units** — nodes with a surface-area-versus-depth relation and real storage volume; they never pressurize.
- **Flow dividers** — nodes that split inflow between a continuation link and a diversion link (cutoff, overflow, tabular, or weir; active under kinematic wave, treated as junctions under dynamic wave).

Links connect nodes. The link types are:

- **Conduits** — pipes and open channels of arbitrary cross-section, described by Manning’s equation and a set of cross-section geometry tables.
- **Pumps** — governed by pump curves (five curve types or an ideal-pump model) with on/off depth hysteresis.
- **Orifices** — bottom or side openings with weir/orifice discharge behavior.
- **Weirs** — transverse, side-flow, V-notch, or trapezoidal weirs with an optional transition to orifice flow under submergence.
- **Outlets** — head or depth rating curves (tabular or functional).

1.3 State variables and internal units

Under flow routing the fundamental state variables are exactly three (Table 1):

Symbol	Description
H	Hydraulic head of water at a node
Q	Flow rate in a link
A	Flow area in a link (inferred from Q and geometry)

Table 1: The three fundamental flow-routing state variables.

All other quantities are derived from these three variables, the external inputs, and the fixed input parameters. Internally the engine carries out all calculations in **feet** (length), **seconds** (time), **cfs** (flow), **ft²** (area), and **ft³** (volume). Project input values in SI (metric) or US units are converted to these internal units at parse time; the conversion factors are

$$\text{Ucf}[\text{LENGTH}] = \begin{cases} 1.0 & (\text{US}) \\ 0.3048 & (\text{SI}) \end{cases}, \quad (2)$$

$$\text{Ucf}[\text{VOLUME}] = \begin{cases} 1.0 & (\text{US}) \\ 0.02832 & (\text{SI}) \end{cases}, \quad (3)$$

(note that the SI volume factor is the legacy *truncated* value, not 0.3048^3 , deliberately preserved for floating-point parity with the legacy SWMM engine). Flow conversion uses $\text{Qcf} = \{1.0, 448.831, 0.64632, 0.02832, 28.317, 2.4466\}$ for cfs/gpm/MGD/cms/LPS/MLD.

1.4 The four routing formulations

The engine selects the flow-routing method with the `FLOW_ROUTING` option. Table 2 summarizes the four methods.

Method	Governing equations	Features
Steady flow	continuity + Manning normal depth	pass-through, no storage/backwater
Kinematic wave	continuity + uniform-flow rating	no backwater, reversal, or surcharge
Dynamic wave (default)	full St. Venant	backwater, losses, reversal, surcharge
Finite volume (explicit)	St. Venant, Godunov/HLLC	transcritical flow, native dry/wet

Table 2: The four 1D routing formulations of the engine.

The dynamic wave is the default and is treated in the greatest depth here. Each routing method is invoked from a common `Router` orchestrator.

2 Hydraulic model components

2.1 Node data

Each node carries static geometric properties and dynamic state (`NodeData.hpp`):

- z_{inv} — invert elevation (ft).
- d_{full} — full depth (depth to the rim / top of storage).
- d_{sur} — maximum allowed surcharge depth above full depth; the flooding limit is $z_{\text{inv}} + d_{\text{full}} + d_{\text{sur}}$.
- A_{pond} — ponded surface area used when the node floods above its rim and ponding is enabled.
- z_{crown} — elevation of the crown of the highest connecting conduit; the surcharge threshold is $y_{\text{crown}} = z_{\text{crown}} - z_{\text{inv}}$.
- deg — number of connecting links; if negative the node is an upstream terminal node.
- V_{full} — volume at full depth.

Dynamic state (updated each routing step):

- d — water depth above the invert (ft);
- $H = z_{\text{inv}} + d$ — hydraulic head (ft);
- V — stored volume (ft³);
- Q_{lat} — total lateral inflow (ft³/s);
- $Q_{\text{in}}, Q_{\text{out}}$ — total inflow and outflow;
- Q_{ovfl} — overflow/flooding rate;
- Q_{loss} — evaporation + seepage/exfiltration loss rate;
- $\Delta Q_{\text{net}}^{t-1}$ — net inflow $Q_{\text{in}} - Q_{\text{out}}$ of the previous step (used for trapezoidal averaging);
- previous-step values $d^{t-1}, V^{t-1}, Q_{\text{lat}}^{t-1}$.

2.1.1 Node surface area and volume functions

The node geometry functions are implemented in `src/engine/hydraulics/Node.cpp`.

Junction/outfall/divider volume: with $\text{MIN_SURFAREA} = 12.566 \text{ ft}^2$ ($\approx 4\pi$, the area of a 4-ft manhole),

$$V(d) = V_{\text{full}} \frac{d}{d_{\text{full}}}, \quad (4)$$

where $V_{\text{full}} = \text{MIN_SURFAREA} \cdot d_{\text{full}}$ unless overridden.

Storage volume:

- Tabular: $V = \text{table}(d \cdot \text{Ucf}[L]) / \text{Ucf}[V]$ with table interpolation;
- Geometric shapes (cylindrical, conical, paraboloid, pyramidal) use the quadratic surface-area relation $A(d) = c + a d + b d^2$, integrated to the cubic

$$V(d) = d \left(c + d \left(\frac{a}{2} + d \frac{b}{3} \right) \right); \quad (5)$$

- Functional storage: power law $A(d) = c + a d^b$ integrated analytically.

Surface area $A_{\text{surf}}(d)$: non-storage nodes return 0 (the minimum-area floor is applied later in the dynamic-wave node update); storage nodes return the table value (extrapolating linearly) or the analytic $A(d) = c + a d + b d^2$ / $A(d) = c + a d^b$.

Ponded area:

$$A_{\text{pond}}(d) = \begin{cases} A_{\text{surf}}(d) & d \leq d_{\text{full}} \text{ or } A_{\text{pond}} = 0, \\ A_{\text{pond}} & d > d_{\text{full}} \text{ (flooded)}. \end{cases} \quad (6)$$

Depth from volume (the inverse of the volume function, used by the kinematic-wave storage update and by reporting): junction nodes use $d = V / \text{MIN_SURFAREA}$; storage nodes invert the tabular/geometric/functional relations, using Newton–Raphson with bisection fallback where no closed form exists.

Maximum outflow (used by the kinematic-wave and steady-flow link inflow gathering):

$$Q_{\text{max}} = Q_{\text{in}} + \frac{V_{\text{old}}}{dt}, \quad Q_{\text{in}} = \min(Q_{\text{in}}, Q_{\text{max}}). \quad (7)$$

2.2 Link data

Each link carries static properties and dynamic state (`LinkData.hpp`). For conduits the per-link (per-barrel) conveyance parameters are (`Link.cpp`, `computeConveyance`):

$$\beta = \frac{\text{PHI} \sqrt{|S_0|}}{n}, \quad \text{PHI} = 1.486 \quad (\text{Manning US factor}), \quad (8)$$

$$\text{rough_factor} = g \left(\frac{n}{\text{PHI}} \right)^2, \quad (9)$$

$$Q_{\text{full}} = S_{\text{full}} \cdot \beta, \quad Q_{\text{max}} = S_{\text{max}} \cdot \beta, \quad (10)$$

where n is the Manning roughness, S_0 the conduit slope, $S(a) = A R^{2/3}$ the *section factor* (see Section 4), $S_{\text{full}} = S(A_{\text{full}})$ and $S_{\text{max}} = \max_a S(a)$.

Dynamic state per link: Q (current flow, $+$ = node1→node2), d (midpoint depth), V (volume), Fr (Froude number), `flow_class` (DRY, UP_DRY, DN_DRY, SUBCRITICAL, SUPERCRITICAL, UP_CRITICAL, DN_CRITICAL), and previous-step values.

2.3 Conduit slope and length

For a conduit connecting node i and node j with end offsets z_1, z_2 , the end invert elevations of the conduit are $Z_1 = z_{\text{inv},1} + z_1$, $Z_2 = z_{\text{inv},2} + z_2$. The conduit slope is

$$S_0 = \frac{\Delta y}{\Delta x}, \quad \Delta y = Z_1 - Z_2, \quad \Delta x = \sqrt{L^2 - \Delta y^2}, \quad (11)$$

with a minimum imposed drop of $|\Delta y| \geq 0.001$ ft (overrideable).

2.4 Conduit lengthening (Courant stability)

If a positive `LENGTHENING_STEP` is supplied, short conduits are artificially lengthened so that the Courant condition for the user-supplied time step is met (`Routing.cpp:135-195`):

$$t_{\text{step}} = \min(\text{routing_step}, \text{lengthening_step}), \quad v_{\text{full}} = \frac{\text{PHI}}{n} \frac{S_{\text{full}}}{\sqrt{|S_0|} A_{\text{full}}}, \quad (12)$$

$$\text{ratio} = \frac{(\sqrt{g y_{\text{full}}} + v_{\text{full}}) t_{\text{step}}}{L}, \quad L' = \max(L, \text{ratio} \cdot L), \quad (13)$$

For open channels y_{full} is replaced by the hydraulic depth $A_{\text{full}}/W_{\text{max}}$. If $L' > L$, the slope and roughness are adjusted to preserve equal head loss at any flow,

$$S'_0 = \frac{|S_0|}{L'/L}, \quad n' = \frac{n}{\sqrt{L'/L}}, \quad (14)$$

and β , `rough_factor`, Q_{full} , Q_{max} are recomputed from the adjusted values. The lengthened length L' (`mod_length`) is used in the momentum equation; the raw length L is used for volume accounting.

3 Simulation orchestration

3.1 The overall time-stepping loop

The engine is a single-step API. Each call to `SWMMEngine::step()` executes exactly one routing step (or one runoff-only step when routing is disabled). The sequence within a step is (`SWMMEngine.cpp:941`):

1. Compute the routing time step dt_{next} . If the variable-step option is enabled, the Courant-limited step is

$$dt_{\text{cfl}} = \text{router.getAdaptiveStep}(\text{routing_step}, \text{variable_step}),$$

then $dt_{\text{next}} = \min(\text{routing_step}, dt_{\text{cfl}})$ clamped to the remaining simulation duration.

2. Save the old hydraulic state (`ctx.save_state()`).
3. Reset the per-step mass-balance accumulators.
4. Run the pipeline:
 - (a) `stepRunoff(dt)` — hydrology (always);
 - (b) if routing is enabled: `stepRouting(dt)` (hydraulics + quality), `updateStatistics(dt)`, `updateRoutingMassBalance(dt)`;
 - (c) `computeFinalStorage()`.
5. Advance the simulation clock and post output snapshots.

3.2 Runoff and routing clocks

The hydrology and hydraulics run on *independent* clocks:

- The **runoff clock** advances at the wet step (300s default) or dry step (3600s default), shortened to align with rain-gage boundaries. `stepRunoff` runs multiple runoff sub-steps within one routing step (while $t_{\text{runoff}} < t_{\text{routing}} + dt$).
- The **routing clock** advances by dt (the routing step) each `step()` call.

After the runoff sub-steps, the wet-weather lateral inflow delivered to nodes is obtained by *linear interpolation* between the bracketing runoff evaluations: with $f = (t_{\text{elapsed}} - t_{\text{runoff,old}}) / (t_{\text{runoff,new}} - t_{\text{runoff,old}})$,

$$Q_{\text{runoff}} = (1 - f) Q_{\text{runoff}}^{\text{old}} + f Q_{\text{runoff}}^{\text{new}}, \quad (15)$$

plus runoff and groundwater contributions. These are scattered into `nodes.runoff_inflow` / `nodes.gw_inflow`.

3.3 Lateral inflows

Every routing step, the lateral inflows are assembled from decomposed source buffers in legacy order (`assembleLateralInflows`, `SWMMEngine.cpp:5691`):

$$Q_{\text{lat}} = Q_{\text{ext}} + Q_{\text{dwf}} + Q_{\text{wet}} + Q_{\text{gw}} + Q_{\text{rdii}} + Q_{\text{iface}} + Q_{\text{user}} + Q_{\text{coupling}}, \quad (16)$$

where the terms are external/inflow time series, dry-weather flow, wet-weather (runoff) inflow, groundwater inflow, RDII, interface-file inflow, user-forced inflow, and 1D-2D coupling inflow respectively.

3.4 The routing step

`Router::step(ctx, dt, evap_rate, non_conduit_fn)` (`Routing.cpp:308`) performs, in order:

1. **initNodeFlows** — initialize each node’s inflow from lateral flow and outflow from losses (storage evaporation and Green-Ampt exfiltration, jointly capped against the stored volume); set overflow from excess stored volume.
2. **computeConduitLosses** — conduit evaporation (open sections) and seepage loss rates. For DYNWAVE this is instead recomputed per Picard iteration inside the solver.
3. **setAllOutfallDepths** — set the outfall boundary stages (Section 13).
4. **Solver dispatch** — KINWAVE, DYNWAVE, FV, or STEADY.
5. **computeDividerFlows** — apply flow-divider logic.
6. **updateLinkStates** — recompute node heads $H = z_{\text{inv}} + d$.

Inside the dynamic-wave solver, the non-conduit callback computes pump/orifice/weir/outlet flows inside the Picard iteration, in link-index order, with immediate scatter into the node inflow/outflow accumulators.

4 Cross-section geometry

4.1 The four geometric relations

For a conduit, the flow area A , top width W , wetted perimeter P_w , and hydraulic radius $R = A/P_w$ are functions of the flow depth y . The engine pre-computes, for each shape, the four fundamental relations (`XSectKernels.hpp`, `xsect_tables.hpp`):

$$A = A(y), \quad W = W(y), \quad R = R(y), \quad (17)$$

$$S = S(A) = A R^{2/3} \quad (\text{section factor, “conveyance shape”}), \quad (18)$$

together with their inverses $y = y(A)$, $A = A(S)$, and the derivative dS/dA . Closed-form expressions exist for the standard shapes (circular, filled circular, rectangular, trapezoidal, triangular, parabolic, power, eggs, horseshoe, gothic, basket-handle, etc.); irregular and custom shapes use tabulated (transect) tables with linear interpolation, and the inverse $A(S)$ is solved by Newton iteration or table inversion.

4.2 Manning and the section factor

Manning's equation in US customary form is

$$Q = \frac{1.486}{n} A R^{2/3} \sqrt{S_f} = \beta \Psi(A) \sqrt{S_f/S_0}, \quad \Psi(A) = A R^{2/3}, \quad (19)$$

where S_f is the friction slope, $\beta = (1.486/n)\sqrt{S_0}$, and $\Psi(A)$ is the section factor. For many closed shapes the section factor peaks below full depth, so the maximum discharge occurs at a less-than-full area.

5 Steady flow routing

Under steady flow (`executeSteadyFlow`, `Routing.cpp:637`), links are processed in topologically sorted order. For each conduit:

1. The upstream node's inflow Q_{in} is gathered (limited by the maximum outflow).
2. The per-barrel flow is $q = Q_{\text{in}}/\text{barrels}$ minus the conduit loss rate, clamped at zero.
3. If $q \geq Q_{\text{full}}$: $q = Q_{\text{full}}$ and $A = A_{\text{full}}$; otherwise the normal-depth area is found from the inverse section factor,

$$s = \frac{q}{\beta}, \quad A = A(s), \quad (20)$$

and the depth from $y = y(A)$.

4. The flow is uniform along the conduit ($Q_{\text{out}} = q \cdot \text{barrels}$, $A_1 = A_2 = A$); the link volume is $V = A L$ barrels.

Non-conduit links simply pass the upstream node's inflow through unchanged. Steady flow converges in one pass.

6 Kinematic wave routing

6.1 Governing equations

The kinematic wave model combines the continuity equation with the uniform-flow rating curve, dropping the inertia, pressure-gradient, and (partially) convective terms of the St. Venant momentum equation. It cannot represent backwater, flow reversal, or pressurization, and requires a directed acyclic network. Starting from the St. Venant equations and setting the bed slope equal to the friction slope, $S_0 = S_f$, the governing pair is:

$$\frac{\partial A}{\partial t} + \frac{\partial Q}{\partial x} = 0 \quad (\text{continuity}), \quad (21)$$

$$Q = \beta \Psi(A), \quad \Psi(A) = A R^{2/3}, \quad \beta = \frac{1.486}{n} \sqrt{S_0} \quad (\text{rating curve}). \quad (22)$$

6.2 Finite-difference scheme

A weighted Wendroff implicit finite-difference scheme discretizes the continuity equation between the upstream (1) and downstream (2) ends:

$$\frac{(1 - \theta)(A_1^{t+1} - A_1^t) + \theta(A_2^{t+1} - A_2^t)}{\Delta t} + \frac{(1 - \varphi)(Q_2^t - Q_1^t) + \varphi(Q_2^{t+1} - Q_1^{t+1})}{L} = 0, \quad (23)$$

with $\theta = \varphi = 0.6$. Because each junction has at most one outlet conduit, processing links in topological order leaves only A_2^{t+1} and Q_2^{t+1} as unknowns; Q_1^{t+1} is known from the upstream node's inflow and A_1^{t+1} from the inverse section factor at Q_1^{t+1}/β . Substituting the rating curve yields the single nonlinear equation

$$f(A_2^{t+1}) = \beta \Psi(A_2^{t+1}) + C_1 A_2^{t+1} + C_2 = 0, \quad (24)$$

with (in the engine's normalized implementation, `KinematicWave.cpp:solveConduit`)

$$C_1 = \frac{L \theta}{\Delta t \varphi}, \quad (25)$$

$$C_2 = \frac{L}{\Delta t \varphi} \left[(1 - \theta)(A_1^{t+1} - A_1^t) - \theta A_2^t \right] + \frac{1 - \varphi}{\varphi} (Q_2^t - Q_1^t) - Q_1^{t+1}. \quad (26)$$

6.3 Root finding

Equation $f(A_2^{t+1}) = 0$ is solved by a bracketed Newton–Raphson iteration (maximum 40 iterations). Because the section factor can have two roots (peak at $A_{\max}$ below full), the bracket is pre-screened: the initial bracket is $[A_{\max}, A_{\text{full}}]$; if both bounds produce the same sign it is reset to $[0, A_{\max}]$. If both bounds give negative f the conduit is full ($Q = Q_{\text{full}}$); if both give positive f the flow is zero.

6.4 Storage nodes under kinematic wave

Storage nodes may have any number of outlet links. Their mass balance is integrated with the trapezoidal rule,

$$V^{t+1} = V^t + \frac{1}{2} (Q_{\text{in}}^t + Q_{\text{in}}^{t+1} - Q_{\text{out}}^t - Q_{\text{out}}^{t+1}) \Delta t, \quad (27)$$

solved by successive approximation ($\omega = 0.55$, convergence tolerance 0.005 ft) because both Q_{out} and V depend on the head. The storage head is then updated once more after all link flows are known (`Routing.cpp:345–389`).

7 Dynamic wave routing: governing equations

7.1 The St. Venant equations

The dynamic wave model solves the full one-dimensional de Saint-Venant equations (continuity and momentum):

$$\underbrace{\frac{\partial A}{\partial t}}_{\text{storage}} + \underbrace{\frac{\partial Q}{\partial x}}_{\text{convective flux}} = 0, \quad (28)$$

$$\underbrace{\frac{\partial Q}{\partial t}}_{\text{local acceleration}} + \underbrace{\frac{\partial}{\partial x} \left(\frac{Q^2}{A} \right)}_{\text{convective acceleration}} + \underbrace{gA \frac{\partial H}{\partial x}}_{\text{pressure gradient}} + \underbrace{gA S_f}_{\text{friction}} = 0, \quad (29)$$

where A is the flow area, Q the discharge, $H = Z + y$ the hydraulic head (Z conduit invert, y depth), g gravity, and S_f the friction slope. The friction slope follows Manning:

$$S_f = \left(\frac{n}{1.486} \right)^2 \frac{Q |U|}{A R^{4/3}}, \quad U = \frac{Q}{A}. \quad (30)$$

7.2 Finite-difference momentum equation

The engine uses an implicit (backwards Euler) finite-difference form. With spatial differences over the conduit length L and temporal differences over Δt , the flow update at each conduit is

$$Q^{t+\Delta t} = \frac{Q^t + \Delta Q_{\text{inertia}} + \Delta Q_{\text{pressure}} + \Delta Q_{\text{loss}}}{1 + \Delta Q_{\text{friction}}}, \quad (31)$$

where all geometric quantities are evaluated at the new time $t + \Delta t$. The four terms are:

$$\Delta Q_{\text{inertia}} = 2\bar{U} (\bar{A}^{t+\Delta t} - \bar{A}^t) + \bar{U}^2 \frac{A_2 - A_1}{L} \Delta t, \quad (32)$$

$$\Delta Q_{\text{pressure}} = -g\bar{A} \frac{H_2 - H_1}{L} \Delta t, \quad (33)$$

$$\Delta Q_{\text{friction}} = g \left(\frac{n}{1.486} \right)^2 \frac{|\bar{U}|}{\bar{R}^{4/3}} \Delta t, \quad (34)$$

$$\Delta Q_{\text{loss}} = \text{local (minor) losses and evap/seepage terms.} \quad (35)$$

7.3 Node continuity

Conservation of volume at each node assembly (the node plus half of each connected link) requires

$$\frac{\partial V}{\partial t} = A_S \frac{\partial H}{\partial t} = \sum Q, \quad (36)$$

with $A_S = A_{SN} + \sum A_{SL}$ the assembly surface area (the node's own storage surface area plus the half-link surface areas contributed by each conduit). In finite-difference form:

$$H^{t+\Delta t} = H^t + \frac{\frac{\Delta t}{2} (\sum Q^t + \sum Q^{t+\Delta t})}{(A_{SN} + \sum A_{SL})^{t+\Delta t}}, \quad (37)$$

i.e. a trapezoidal update of the head using the *average* of the net flows at t and $t + \Delta t$.

8 The dynamic wave solution algorithm

8.1 Overview: the Picard (successive-approximation) iteration

Equations (31) and the node continuity are solved implicitly over each time step by *functional iteration* (Picard iteration / successive approximations). The steps, matching the engine's `DWSolver::execute` (`DynamicWave.cpp:1026`), are:

1. **initNodeStates** — reset per-node accumulators: inflow = 0, outflow = losses + negative lateral flow, inflow + positive lateral flow; surface area from the storage/ponding geometry; converged = 0, $\sum dQ/dH = 0$.
2. **computeLinkGeometry** — batch-compute the end depths y_1, y_2 , midpoint depth $\bar{y}$, top widths, areas, and hydraulic radii of all conduits from the current node heads; classify each conduit's flow regime; apply Preissmann-slot overrides where surcharged.
3. **Momentum kernels** — solve the implicit momentum equation for each conduit (Section 9), commit the new flows, and scatter them to the node inflow/outflow accumulators, accumulating $\sum dQ/dH$ and the surface-area contributions.
4. **Non-conduit structures** — compute pump/orifice/weir/outlet flows (Section 12).
5. **Outfall depths** — re-set outfall boundary stages from the current flows.
6. **Node depth update** — solve the continuity equation for each node head (Section ??); tally the unconverged nodes.

7. If any node changed by more than the head tolerance and the trial limit has not been reached, mark links whose both end nodes converged as *bypassed* (their flows are held), and iterate.

Under-relaxation: after the first iteration every updated flow and head is relaxed with $\omega = 0.5$,

$$x^{k+1} = (1 - \omega) x^k + \omega x_{\text{raw}}^{k+1}, \quad \omega = 0.5, \quad (38)$$

and a sign change in flow is forced through a small nonzero value ($q = \pm 0.001$).

8.2 Convergence

A node is converged when both the raw Picard residual and the accepted (possibly Anderson-mixed) movement are within the head tolerance $\varepsilon_H = 0.005$ ft (default). The routing step converges when no non-outfall node is unconverged. The maximum number of trials is 8 by default (MAX_TRIALS); a step that converges on its last allowed trial is still counted as converged.

8.3 Flow classification

During geometry computation each conduit is classified into one of seven flow classes (DRY, UP_DRY, DN_DRY, SUBCRITICAL, SUPERCRITICAL, UP_CRITICAL, DN_CRITICAL) based on the end depths and the head relative to the end invert elevations, using the normal depth y_n (from the inverse section factor at Q/β) and critical depth y_c (Section 13). The class drives:

- the surface-area adjustments for dry/critical ends (Table 3),
- the depth used at the critical end (replaced by $\min(y_n, y_c)$),
- the friction-based normal-flow limit.

Condition	Criteria	Adjustment
Upstream dry	$y_1 = 0; Z_1 > E_1$	$A_{SL1} = 0$ if $H_2 \leq Z_1$, else critical adjustment
Downstream dry	$y_2 = 0; Z_2 > E_2$	$A_{SL2} = 0$ if $H_1 \leq Z_2$, else critical adjustment
Upstream critical	$Q < 0; Z_1 > E_1; H_1 - Z_1 < y_*$	$y_1 = y_*$; $A_{SL1} = 0$
Downstream critical	$Q > 0; Z_2 > E_2; H_2 - Z_2 < y_*$	$y_2 = y_*$; $A_{SL2} = 0$

Table 3: Surface-area and depth adjustments for dry/critical conduit ends ($y_* = \min(y_n, y_c)$; E = node invert, Z = conduit invert).

8.4 Surface-area contributions

For a subcritical conduit, the free-surface areas contributed to its two end nodes are

$$A_{SL1} = \frac{W_1 + \bar{W}}{2} \frac{L}{2}, \quad A_{SL2} = \frac{\bar{W} + W_2}{2} \frac{L}{2} \text{fasnh}, \quad (39)$$

where $W_1, W_2, \bar{W}$ are the top widths at $y_1, y_2, \bar{y}$, and fasnh is a factor between normal and critical depth for a nearly critical downstream end. For dry/critical ends the half-link surface area is adjusted per Table 3; a completely dry conduit contributes $A_{SL1} = A_{SL2} = \text{FUDGE} \cdot L/2$. A closed conduit whose midpoint depth exceeds the crown cutoff ($y/y_{\text{full}} \geq 0.96$ under EXTRAN) uses a width capped at the crown in the surface-area computation, providing the small nonzero contribution that keeps the node Picard denominator bounded.

9 The link momentum kernel

Each conduit is solved by one of the momentum kernels in `DynamicWave.cpp` (`processManningLink`, `processDryLink`, `processForceMainLink`), dispatched by momentum category. This section transcribes the Manning (free-surface and closed-conduit) kernel, which is the main one.

9.1 Velocity, Froude number, and inertial damping

With $\bar{A}$ the midpoint area and q_{last} the per-barrel flow from the previous iteration,

$$v = \frac{q_{\text{last}}}{\bar{A}}, \quad |v| \leq V_{\text{max}} = 50 \text{ ft/s} \quad (\text{velocity cap}). \quad (40)$$

For a non-surcharged conduit the Froude number uses the hydraulic depth $\bar{A}/\bar{W}$:

$$\text{Fr} = \frac{|v|}{\sqrt{g \bar{A}/\bar{W}}} \quad (0 \text{ for a closed conduit within FUDGE of full}). \quad (41)$$

The inertial-damping factor follows the *local partial inertia* approach (linear blend over $0.5 \leq \text{Fr} \leq 1$):

$$\sigma = \begin{cases} 1 & \text{Fr} \leq 0.5, \\ 2(1 - \text{Fr}) & 0.5 < \text{Fr} < 1, \\ 0 & \text{Fr} \geq 1, \end{cases} \quad \text{i.e. } \sigma = \text{clamp}(2(1 - \text{Fr}), 0, 1). \quad (42)$$

The `INERTIAL_DAMPING` option overrides this: `NONE` forces $\sigma = 1$, `FULL` forces $\sigma = 0$ (no inertial terms at all); a closed conduit that is surcharged always has $\sigma = 0$.

9.2 Upstream weighting

The pressure and friction terms use *upstream-weighted* average area and hydraulic radius, reflecting that supercritical flow is influenced only by upstream conditions. With the weighting factor

$$\rho = \begin{cases} \sigma & q_{\text{last}} > 0 \text{ and } H_1 \geq H_2 \text{ (and not full)}, \\ 1 & \text{otherwise}, \end{cases} \quad (43)$$

$$\bar{A}' = A_1 + \rho(\bar{A} - A_1), \quad \bar{R}' = R_1 + \rho(\bar{R} - R_1). \quad (44)$$

9.3 The six momentum terms

The engine evaluates the momentum equation as the six contributions $dq_1 \dots dq_6$ (per barrel):

$$dq_1 = \Delta t \cdot \text{rough_factor} \cdot \frac{|v|}{\bar{R}'^{4/3}}, \quad (45)$$

$$dq_2 = \Delta t g \bar{A}' \frac{H_2 - H_1}{L}, \quad (46)$$

$$dq_3 = 2 v \sigma (\bar{A}^{t+\Delta t} - \bar{A}^t), \quad (47)$$

$$dq_4 = \Delta t v^2 \sigma \frac{A_2 - A_1}{L}, \quad (48)$$

$$dq_5 = \frac{\Delta t}{2L} \left[K_{\text{in}} \frac{|q|}{A_1} + K_{\text{out}} \frac{|q|}{A_2} + K_{\text{avg}} \frac{|q|}{\bar{A}} \right], \quad (49)$$

$$dq_6 = \frac{2.5 \Delta t v}{L} (Q_{\text{evap}} + Q_{\text{seep}}), \quad (50)$$

where $\text{rough_factor} = g(n/1.486)^2$, the local-loss coefficients $K_{\text{in}}, K_{\text{out}}, K_{\text{avg}}$ are the user's inlet, outlet, and average (loss-weighted) minor-loss coefficients, and $Q_{\text{evap}} + Q_{\text{seep}}$ is the per-conduit loss rate.

The flow update (per barrel) and its head gradient are

$$q = \frac{q_{\text{old}} - dq_2 + dq_3 + dq_4 + dq_6}{1 + dq_1 + dq_5}, \quad (51)$$

$$\frac{dQ}{dH} = \frac{1}{1 + dq_1 + dq_5} \frac{g \Delta t \bar{A}'}{L} \text{ barrels.} \quad (52)$$

The dQ/dH gradient feeds the node surcharge solver (Section 10).

9.4 Flow limiting and post-processing

After the raw momentum update, `applyFlowLimits` (`DynamicWave.cpp:2211`) applies, in order:

1. **Culvert inlet control** (FHWA HEC-5): if a culvert code is present and the conduit is not full, $q \leftarrow \min(q, q_{\text{inlet}})$.
2. **Normal-flow limit**: for an open/free-surface conduit not full, if the slope condition ($y_1 < y_2$, i.e. the water-surface slope is smaller than the bed slope) or the upstream-Froude condition ($\text{Fr}_1 \geq 1$) holds (per the `NORMAL_FLOW_LIMITED` option), then

$$q \leftarrow \min\left(q, \beta A_1 R_1^{2/3}\right), \quad (53)$$

the Manning normal flow at the upstream depth.

3. **Under-relaxation** (iterations > 0): $q = (1 - \omega)q_{\text{last}} + \omega q$ with a sign-change clamp to ± 0.001 .
4. **User flow limit**: $|q| \leq q_{\text{limit}}$.
5. **Flap gates**: reverse flow through a flap gate is zeroed.
6. **Dry-node check**: flow is forced to $\pm \text{FUDGE}$ if the upstream (downstream) node is dry.

Finally,

$$Q^{t+\Delta t} = q \text{ barrels}, \quad d_{\text{link}} = \min(\bar{y}, y_{\text{full}}), \quad V_{\text{link}} = \frac{A_1 + A_2}{2} L_{\text{raw}} \text{ barrels.} \quad (54)$$

9.5 Force mains

Force mains (a `FORCE_MAIN` cross-section, always full) use a separate kernel (`processForceMainLink`) with $\sigma = 0$ and friction from either the Hazen–Williams or Darcy–Weisbach equation,

$$\begin{aligned} dq_1 &= \Delta t g \frac{S_f}{|v|}, \\ q &= \frac{q_{\text{old}} - dq_2 + dq_6}{1 + dq_1 + dq_5}. \end{aligned} \quad (55)$$

10 Node continuity, flooding, and surcharge

10.1 Net flow and volume change

At each node, per Picard iteration (`DynamicWave.cpp`, function `setNodeDepth`), the net flow and trapezoidal volume change are:

$$\Delta Q = Q_{\text{in}} - Q_{\text{out}}, \quad \Delta V = \frac{1}{2} (\Delta Q_{\text{net}}^{t-1} + \Delta Q) \Delta t, \quad (56)$$

where $\Delta Q_{\text{net}}^{t-1}$ is the net flow of the previous step (trapezoidal/Crank–Nicolson averaging).

10.2 Free-surface update (EXPLICIT continuity)

For a node that is not surcharged, the depth update is

$$\Delta y = \frac{\Delta V}{A_S}, \quad y^{t+\Delta t} = y^t + \Delta y, \quad (57)$$

where the assembly surface area is floored by the minimum surface area,

$$A_S = \max \left(A_{SN} + \sum A_{SL}, A_{\min} \right), \quad A_{\min} = 12.566 \text{ ft}^2, \quad (58)$$

the default (overrideable via the `MIN_SURFAREA` option). The minimum area is a purely computational device; it does not add volume.

10.3 Surge (EXTRAN) update

A node is surcharged when all conduits connected to it are full or its head exceeds the crown of its highest conduit ($y > y_{\text{crown}}$), and it is not a storage or outfall node and is not ponding. Under EXTRAN the continuity equation becomes $\sum Q = 0$, enforced through a perturbation (Newton/Hardy Cross) form. The surcharged depth update is

$$\Delta y = \frac{\alpha \Delta Q}{\text{denom}}, \quad y^{t+\Delta t} = y^t + \Delta y, \quad (59)$$

with the terminal-node correction $\alpha = 0.6$ for upstream terminal nodes (only outflow links, $\text{deg} < 0$) and 1 otherwise, and a crown-proximity blend for a smooth transition into/out of surcharge:

$$\text{denom} = \sum \frac{dQ}{dH} + \left(\frac{A_S^{\text{old}}}{\Delta t} - \sum \frac{dQ}{dH} \right) e^{-15f}, \quad f = \frac{y^t - y_{\text{crown}}}{y_{\text{crown}}}, \quad (60)$$

applied while $y^t < 1.25 y_{\text{crown}}$. The head is kept at or above the crown ($y \geq y_{\text{crown}} - \text{FUDGE}$). The $\sum dQ/dH$ accumulated at each node is the sum of the per-link gradients (52); note the engine accumulates $\sum dQ/dH$ as a positive magnitude with $dQ_{\text{net}}/dH = -\sum dQ/dH$ (a head rise increases net outflow).

10.4 Semi-implicit node continuity

The `NODE_CONTINUITY` option (default EXPLICIT) can select the unified *semi-implicit* formulation, a single smooth expression valid for both free-surface and surcharged states. Linearizing the net flow about the current head estimate with the flow gradients, $\sum Q^{t+\Delta t} \approx \sum Q + \sum (dQ/dH) \Delta H$, and substituting into the trapezoidal head update yields

$$\Delta y = \frac{\Delta V}{A_S + \frac{\Delta t}{2} \sum \frac{dQ}{dH}}, \quad y^{t+\Delta t} = y^t + \Delta y, \quad (61)$$

with the denominator floored by $A_{\min}$. The flow-gradient term damps the update (a head rise that increases net outflow reduces the net fill), and the expression is C^1 -smooth through the crown, which is what makes it compatible with Anderson acceleration.

10.5 Flooding and ponding

The maximum non-flooded depth is $y_{\text{max}} = d_{\text{full}} + d_{\text{sur}}$ (for non-ponding nodes; ponding nodes are never capped). If the candidate depth exceeds y_{max} :

- **Non-ponding:** the head is capped at $y_{\max}$ and the excess is lost as overflow,

$$Q_{\text{ovfl}} = \frac{\Delta V}{\Delta t}, \quad V = V_{\text{full}}; \quad (62)$$

- **Ponding** (with `ALLOW_PONDING` and a positive ponded area): the ponded node acts as a constant-area storage node above the rim,

$$V = \max(V^{\text{old}} + \Delta V, V_{\text{full}}), \quad Q_{\text{ovfl}} = \frac{V - \max(V^{\text{old}}, V_{\text{full}})}{\Delta t}, \quad (63)$$

and the head is free to rise above the rim.

A ponded node is not allowed to drop much below full depth once ponded ($y \geq d_{\text{full}} - \text{FUDGE}$).

11 Surcharge methods

The `SURCHARGE_METHOD` option selects how pressurized flow in closed conduits is treated. All three methods update the flow area of a surcharged conduit (depth $> y_{\text{full}}$) by adding a narrow slot of width w_s :

$$A = A_{\text{full}} + (y - y_{\text{full}}) w_s, \quad (64)$$

while the hydraulic radius is clamped to its full value R_{full} (the slot contributes storage but not friction).

11.1 EXTRAN (default)

The classical perturbation method of Section 10: dQ/dH -based surcharged node updates with a crown cutoff at $y/y_{\text{full}} = 0.96$ used for the width used in the surface-area computation.

11.2 Static Preissmann slot

The conduit carries a fictitious narrow slot above the crown so the free-surface formulation remains valid throughout pressurization. The slot width follows the Sjöberg (1982) formula,

$$\frac{w_s}{W_{\text{max}}} = 0.5423 \exp\left(-\left(\frac{y}{y_{\text{full}}}\right)^{2.4}\right), \quad (65)$$

applied for $0.985257 \leq y/y_{\text{full}} \leq 1.78$, clamped at $w_s = 0.01 W_{\text{max}}$ above that, and zero below the cutoff.

11.3 Dynamic Preissmann slot

An OpenSWMM extension (Sharior, Hodges & Vasconcelos 2023) in which the slot area evolves in time as an element of transient storage. The slot top width is driven by a target pressure-wave celerity c_{pT} and a time-varying Preissmann number P :

$$T_s = \frac{g A_{\text{full}}}{c_{pT}^2} P^2, \quad P_0 = \max\left(\frac{c_{pT}}{\alpha c_g}, 1\right), \quad c_g = \sqrt{g A_{\text{full}}/W_{\text{max}}}, \quad (66)$$

with the shock parameter $\alpha = 3$ (default). The accumulated slot area is path-dependent,

$$A_s \leftarrow \max(A_s + T_s \Delta h_s, 0), \quad h_s = \max(\bar{y} - y_{\text{full}}, 0), \quad (67)$$

and P decays exponentially after pressurization onset and is spatially smoothed across nodes. While a slot is active, the top width is T_s and the surface area contributed to a surcharged end node is $T_s L/4$. Nodal heads are updated with the ordinary free-surface formula at all times; the surcharge branch is never invoked. The variable time step uses the pressure celerity $c_p = c_{pT}/P$.

12 Non-conduit hydraulic structures

The non-conduit structure flows are computed by `HydStructures.cpp` (`StructureSolver`) inside the dynamic-wave Picard iteration, one link at a time in link-index order (so that two pumps sharing a wet well see each other's draw), with immediate scatter into the node accumulators and under-relaxation at $\omega = 0.5$ for iterations > 0 (pumps exempt).

12.1 Pumps

Pump on/off control uses depth hysteresis applied once per routing step:

$$\text{shut off if } d < d_{\text{off}} \text{ (setting } > 0), \quad \text{start if } d > d_{\text{on}} \text{ (setting } = 0). \quad (68)$$

The pump curve types (1–5) determine the discharge q from the curve versus wet-well volume (type 1), depth (types 2 and 4), or head (types 3 and 5, where q is scaled by the speed setting s and the head by s^2); an ideal pump discharges exactly the upstream node's inflow plus overflow. Flow limiting (legacy `getModPumpFlow`) prevents pumping more than is available (dry-draw protection) and, for type-1 pumps or storage-fed pumps, caps q by $Q_{\text{in}} + V_{\text{old}}/\Delta t$.

12.2 Orifices

Bottom and side orifices use a coefficient and a critical depth set up from the opening height $h_{\text{open}} = s y_{\text{full}}$ (setting s):

$$f = \min\left(\frac{\text{head}}{h_{\text{crit}}}, 1\right), \quad h_{\text{crit}} = \frac{C_d}{0.414} \frac{h_{\text{open}}}{4} \text{ (circular) or } \frac{C_d}{0.414} \frac{h_{\text{open}} W_{\text{max}}}{2(h_{\text{open}} + W_{\text{max}})} \text{ (rectangular)}, \quad (69)$$

$$q = \begin{cases} C_w f^{3/2} & f < 1 \quad \text{(weir-like partial flow)}, \\ C_d A_{\text{eff}} \sqrt{2gH} & f \geq 1 \quad \text{(full orifice)}, \end{cases} \quad (70)$$

with $C_w = C_d \sqrt{h_{\text{crit}}} A_{\text{eff}} \sqrt{2g}$ (sharp-crested weir coefficient 0.414), A_{eff} the cross-section area at the opening height, and H the appropriate head (differential head when submerged). The flow gradient is

$$\frac{dQ}{dH} = \frac{3}{2} \frac{q}{f h_{\text{crit}}} \quad \text{(weir regime)}, \quad \frac{dQ}{dH} = \frac{q}{2H} \quad \text{(orifice regime)}. \quad (71)$$

An optional ARMCO flap-gate head loss and the Villemonte submergence correction may be applied.

12.3 Weirs

The four weir types (transverse, side-flow, V-notch, trapezoidal) discharge according to

$$\begin{aligned} q &= C_d L h^{3/2} && \text{(transverse),} \\ q &= C_d L^{0.83} h^{1.67} && \text{(side-flow, corrected),} \\ q &= C_d s_h h^{5/2} && \text{(V-notch),} \end{aligned} \quad (72)$$

with end contractions reducing the effective length. When the upstream hydraulic grade line reaches the crown ($H_1 \geq H_{\text{crown}}$) and surcharging is allowed, the weir transitions to orifice flow using an equivalent-orifice coefficient computed from the weir discharge at full opening:

$$C_{\text{sur}} = \frac{Q_{\text{weir}}(s y_{\text{full}})}{\sqrt{(s y_{\text{full}})/2}}, \quad q = C_{\text{sur}} \sqrt{H_{\text{orif}}}, \quad (73)$$

with H_{orif} the head to the mid-point of the opening (or the differential head when submerged). If surcharging is not allowed, the head is capped at the crown and the weir equation is kept. Villemonte submergence correction applies when the downstream HGL exceeds the crest. The weir's surface-area contribution is zeroed for SWMM-4 compatibility.

12.4 Outlets

Outlets use head- or depth-based rating curves (tabular or functional):

$$q = C H^e \quad (\text{functional}), \quad q = \text{table}(H) \quad (\text{tabular}), \quad (74)$$

scaled by the setting. The dQ/dH is deliberately left zero (matching legacy), which keeps the node surcharge denominator from being inflated.

13 Outfall boundary conditions

Outfall stages are set at the start of each routing step and again inside each Picard iteration from the current conduit flows. For the conduit connected to an outfall at end offset z , with per-barrel flow q , the normal depth y_n (from the inverse section factor) and critical depth y_c are computed (y_c has closed forms for standard shapes and a numerical root find otherwise). The stage depends on the boundary-condition type (`Outfall.cpp:setOutfallDepth`):

- **FREE**: free overfall, $d = z + \min(y_n, y_c)$.
- **NORMAL**: $d = z + y_n$.
- **FIXED, TIDAL, TIMESERIES**: the stage s_{stage} is constant, from a tidal table (hour of day), or from a time series; then

$$d = \begin{cases} s_{\text{stage}} - z_{\text{inv}} & z + y_c + z_{\text{inv}} < s_{\text{stage}}, \\ \max(0, s_{\text{stage}} - z_{\text{inv}}) & z > 0 \wedge s_{\text{stage}} < z_{\text{inv}} + z, \\ z + y_c & z > 0 \wedge s_{\text{stage}} \geq z_{\text{inv}} + z, \\ y_c & z = 0. \end{cases} \quad (75)$$

The last case keeps a free-discharging fixed outfall at the conduit critical depth. A flap gate at an outfall blocks reverse flow.

14 Flow dividers

Dividers are active only under kinematic-wave (and steady) routing; under dynamic wave they act as ordinary junctions. Given the total inflow Q_{in} at the divider node:

- **Cutoff**: diverts all inflow above a minimum, $Q_{\text{div}} = \max(0, Q_{\text{in}} - q_{\text{min}})$.
- **Overflow**: diverts the inflow above the capacity of the continuation link.
- **Tabular**: $Q_{\text{div}} = Q_{\text{in}} \cdot f(Q_{\text{in}})$ with the fraction from a rating table.
- **Weir**: $Q_{\text{div}} = C_d W d^{3/2}$ from a weir relation on the node depth, capped at the inflow.

The diverted flow is written to the diversion link; the continuation link carries the remainder.

15 Adaptive time stepping and the Courant condition

15.1 Courant–Friedrichs–Lewy condition

Because the dynamic-wave scheme updates flows and heads element by element (there is no simultaneous spatial coupling), it is subject to the Courant condition: the time step must not exceed the time for a dynamic wave to traverse the shortest conduit,

$$\Delta t \leq \frac{L}{|U| + c}, \quad c = \sqrt{g \frac{A}{W}} \quad (\text{gravity-wave celerity}). \quad (76)$$

15.2 Link-based step

For each conduit (`getLinkStep`, `DynamicWave.cpp:3711`) with per-barrel flow $q = |Q|/\text{barrels}$, midpoint area A , and Froude number Fr :

$$t = \frac{V/\text{barrels}}{q} \frac{L'}{L} \frac{\text{Fr}}{1 + \text{Fr}}, \quad (77)$$

(the classic SWMM CFL expression, scaled by the lengthened/raw length ratio). Under the dynamic Preissmann slot a surcharged conduit instead uses the pressure celerity,

$$t = \frac{L(L'/L)}{|v| + c_p}, \quad c_p = \frac{c_{pT}}{P}. \quad (78)$$

15.3 Node-based step

For each non-outfall node below the crown with a nonzero depth-change rate $\dot{y} = |\Delta y/\Delta t|$:

$$t = 0.25 \frac{y_{\text{crown}}}{\dot{y}}, \quad (79)$$

guarding against excessive head change in one step.

15.4 Combining the constraints

The next routing step (`getRoutingStep`) is the minimum over all links, nodes, and virtual-junction pairs, each scaled by the user's Courant factor, then floored and clamped:

$$\Delta t_{\min}^{\text{eff}} = \max\left(\min_{j,i}\{t_j, t_i\} \cdot \text{Cr}, \max(\min(\Delta t_{\min}, \Delta t_{\text{routing}}), 0.001 \text{ s})\right), \quad (80)$$

rounded to milliseconds. The effective step is then capped by the user's fixed `ROUTING_STEP` and the remaining simulation duration. The initial step is the minimum step. The first call (no flows yet) also returns the minimum step. The FV solver substeps internally at its own CFL limit, so the routing step under FV is only a reporting cadence.

16 Stability, convergence, and mass balance

16.1 Stability indicators

Numerical instability appears as non-damping oscillations in flow and water surface, and nodes that repeatedly dry up. Two report metrics quantify it:

- the overall **flow continuity error** (below), and
- the **link Flow Instability Index (FII)** — the normalized count of times a link's flow exceeds both its neighbors.

16.2 Convergence of the Picard loop

A routing step “does not converge” only when the head tolerance ($\varepsilon_H = 0.005 \text{ ft}$) is not met at some non-outfall node after `MAX_TRIALS` iterations. Anderson acceleration (optional) speeds convergence by blending the two most recent operator outputs at each node:

$$\alpha_k = \text{clamp}\left(\frac{r_k(r_k - r_{k-1})}{(r_k - r_{k-1})^2}, 0, 1\right), \quad H_{k+1} = (1 - \alpha_k)G(H_k) + \alpha_k G(H_{k-1}), \quad (81)$$

with $r_k = G(H_k) - H_k$ the residual and G the head-update operator. It is skipped at nodes where G is non-smooth (EXTRAN-surcharged, dynamic-slot active, near the static-slot cutoff, weir/orifice at crown, pump ends, ponding boundary).

16.3 Routing mass balance

The routing continuity error (fraction of total inflow) is

$$\varepsilon_{\text{routing}} = \frac{Q_{\text{dw}} + Q_{\text{wet}} + Q_{\text{gw}} + Q_{\text{rdii}} + Q_{\text{ext}} + V_{\text{init}} - (Q_{\text{flood}} + Q_{\text{out}} + Q_{\text{evap}} + Q_{\text{seep}} + V_{\text{final}})}{Q_{\text{total in}}}, \quad (82)$$

where the inflows are dry-weather, wet-weather, groundwater, RDII, and external, and the outflows are flooding, system outflow, evaporation, seepage, and final storage. Flooding is counted only when the node volume does not exceed its full volume; outfall discharge counts as system outflow, and outfall backflow counts as external inflow. Conduit and storage-node evap/seep losses are charged here as well.

17 Options and defaults

Table 4 summarizes the routing-related options (`SimulationOptions.hpp`) and their defaults.

Option	Default	Purpose
FLOW_ROUTING	DYNWAVE	routing formulation
ROUTING_STEP	20 s	fixed routing step
MINIMUM_STEP	0.5 s	floor for the CFL step
DRY_STEP / WET_STEP	3600 / 300 s	runoff step bounds
VARIABLE_STEP	0.75	Courant factor (0 = fixed step)
LENGTHENING_STEP	0	Courant lengthening step
MAX_TRIALS	8	Picard trial limit
HEAD_TOLERANCE	0.005 (project units)	node convergence tolerance
SURCHARGE_METHOD	EXTRAN	EXTRAN / SLOT / DYNAMIC_SLOT
NODE_CONTINUITY	EXPLICIT	two-branch vs semi-implicit
INERTIAL_DAMPING	PARTIAL	Froude damping policy
NORMAL_FLOW_LIMITED	BOTH	slope/Froude normal-flow cap
MIN_SURFAREA	12.566 ft ²	nodal area floor
ALLOW_PONDING	false	allow ponding above the rim
ANDERSON_ACCEL	false	Anderson acceleration
SKIP_STEADY_STATE	false	skip steady routing

Table 4: Routing-related options and defaults.

18 References

- [1] Rossman, L. A. (2017). *Storm Water Management Model Reference Manual Volume II – Hydraulics*. U.S. EPA.
- [2] OpenSWMM engine source:
 - `src/engine/hydraulics/Routing.cpp`
 - `src/engine/hydraulics/DynamicWave.cpp`
 - `src/engine/hydraulics/KinematicWave.cpp`
 - `src/engine/hydraulics/HydStructures.cpp`

- `src/engine/hydraulics/Node.cpp`
`src/engine/hydraulics/Link.cpp`
`src/engine/hydraulics/Outfall.cpp`
`src/engine/hydraulics/Divider.cpp`
 - `src/engine/hydraulics/XSectBatch.cpp`
`src/engine/hydraulics/XSectKernels.hpp`
`src/engine/core/SimulationOptions.hpp`
`src/engine/core/SWMMEngine.cpp`
- [3] Sharior, S., Hodges, B. R., & Vasconcelos, J. G. (2023). Generalized, dynamic, and transient-storage form of the Preissmann slot. *Journal of Hydraulic Engineering*, 149(11).
- [4] Sjöberg, A. (1982). Sewer network models dagnum and DIVISION – a brief description. *Swedish Water and Wastewater Works Association*.
- [5] Roesner, L. A., Nichandros, H. M., Shubinski, R. P., Feldman, A. D., Abbott, J. W., & Delleur, J. W. (1981). *Physical model for combined sewer overflows*. Proc. ASCE.
- [6] de Almeida, G. A. M., & Bates, P. D. (2013). Applicability of the local inertial approximation of the shallow water equations to flood modeling. *Water Resources Research*, 49(8).